Juego de aeronaves

Documentación del proyecto

Avance sobre la plantilla académica del curso de Dinámica de Sistemas

Facultad de Ciencias de la Ingeniería
Escuela Profesional de Ingeniería de Software y Sistemas

Dato	Detalle
Curso	Dinámica de Sistemas
Docente	M. Sc. Juan Carlos Pinto L. — según la plantilla proporcionada
Equipo	Pendiente: nombres de integrantes y asignación de responsabilidades
Fecha de revisión	26 de octubre de 2026
Versión	0.1 — avance sustentado en inspección de código
Repositorio	slim-carrasco/juego-de-aviones
Copia local revisada	/home/joel/Work/juego-de-aviones
Base Git	074a2ccd393f2a0aa2ab7339544d0f1fd24cec2c; incluye cambios locales sin confirmar

Criterio de elaboración

Se conserva la estructura temática de la plantilla y se completa con el estado de la copia local del proyecto. “Implementado” significa localizado en el código; la validación mediante ejecución se informa por separado. Los diagramas se elaboraron a partir de clases, métodos y tablas existentes.

No se atribuyen mediciones ni pruebas aprobadas sin evidencia de ejecución. Los datos personales, las capturas de Windows y la validación del docente permanecen pendientes.

Organización del documento

Resumen e introducción; objetivos y visión; modelo de sistema; variables y relaciones causales; modelo dinámico e indicadores; Sprint 0; requisitos; diagramas UML y entidad-relación; referencias y entregables; cinco anexos; estado de validación.

Resumen e introducción

Resumen

El proyecto Juego de aeronaves desarrolla una aplicación de escritorio en C# y Windows Forms para practicar conceptos de ingeniería de software, programación orientada a objetos y dinámica de sistemas. Incluye un modo individual, un duelo local para dos personas y una modalidad de combate en red de tres jugadores contra tres. En esta última, un servidor ASP.NET Core mantiene la autoridad sobre el movimiento, los disparos, las colisiones, la vida y la condición de victoria. Los clientes envían controles y representan el estado recibido mediante WebSocket. La solución separa interfaz, reglas de juego, contratos, persistencia y servidor. MySQL conserva cuentas, resultados y estadísticas; las contraseñas se almacenan mediante PBKDF2 con sal. El análisis de sistemas identifica la espera de jugadores, la formación de salas, la evolución de la vida y la generación de resultados como procesos relacionados. Este avance documenta objetivos, variables, relaciones causales, requisitos, planificación y modelos de diseño a partir del código local. La programación del servidor establece un objetivo de treinta actualizaciones por segundo y limita a 1024 bytes el contenido JSON de cada cuadro periódico. Estas configuraciones requieren validación de rendimiento bajo carga. Permanecen pendientes la gestión de grupos preformados exigida por la especificación, las capturas de ejecución y las pruebas de integración y aceptación.

Introducción

Un combate aéreo multijugador combina entradas humanas, simulación, comunicación y almacenamiento. Cada acción puede modificar la posición de un avión, producir un proyectil, reducir la vida de un rival y cambiar el desenlace de la partida. La coordinación de estos cambios permite estudiar tanto la construcción del software como la evolución de un sistema en el tiempo.

La plantilla académica proporcionada solicita objetivos, modelado, planificación Scrum, requisitos y diagramas. El presente avance vincula esos apartados con la implementación disponible y con la especificación “Documentación JUEGO DE AERONAVES.docx.pdf”.

Alcance y estado revisado

La interfaz se ejecuta en Windows con .NET 10. El servidor ofrece HTTP para cuentas y resultados y WebSocket para las salas en red. La persistencia utiliza MySQL mediante Entity Framework Core. La revisión incluye archivos modificados y nuevos todavía no incorporados a un commit; por ello no describe únicamente el último estado publicado en GitHub.

Los modos Solo y Duelo local calculan el combate en el cliente mediante FlightGame. La modalidad 3v3 calcula el combate en el servidor mediante TeamBattle. La autoridad del servidor sobre la simulación se aplica al modo en red.

Objetivos y visión del producto

Objetivo general

Aplicar procesos, técnicas y herramientas de ingeniería de software en el desarrollo de un juego de combate aéreo que permita la interacción simultánea de múltiples clientes dentro de una simulación en tiempo real, utilizando una arquitectura cliente-servidor y persistencia de resultados.

Objetivos específicos

1. Definir los requisitos funcionales y de calidad del juego y mantener su trazabilidad con componentes y pruebas.

2. Separar la interfaz, las reglas de simulación, la comunicación y el almacenamiento en proyectos con responsabilidades identificables.

3. Implementar registro, inicio de sesión y control de acceso a las salas mediante sesiones.

4. Coordinar salas con tres jugadores por equipo y validar movimiento, disparos, colisiones y finalización desde el servidor.

5. Conservar en MySQL el resultado del combate y las estadísticas de los seis participantes.

6. Modelar variables, flujos y retroalimentaciones del sistema y establecer indicadores para su evaluación.

7. Validar reglas de negocio y comportamiento de red mediante pruebas automatizadas y escenarios de integración.

Visión del producto

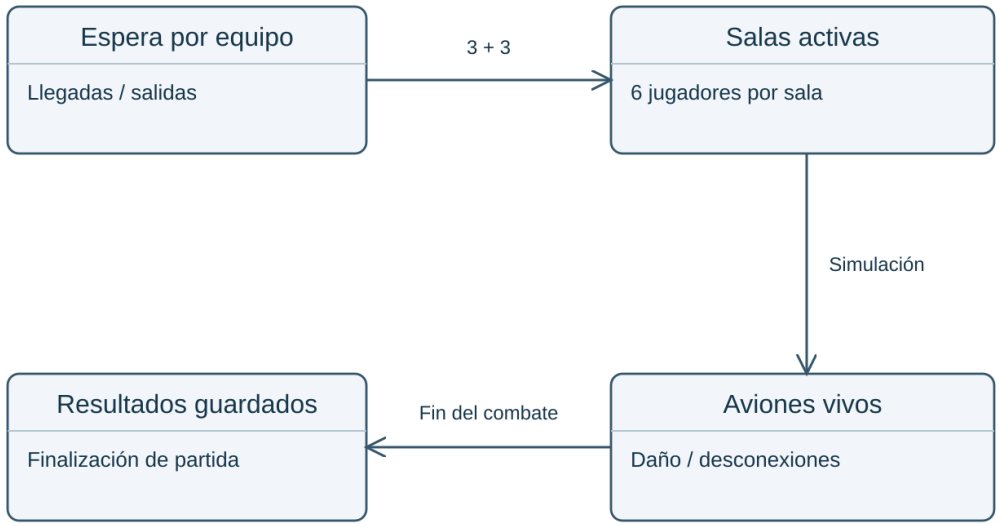
Para estudiantes y docentes que buscan un entorno lúdico y un caso práctico de ingeniería de software, Juego de aeronaves ofrece combates individuales, locales y por equipos. En red, cada participante elige un avión y un equipo, espera a completar la sala y controla su aeronave mientras el servidor mantiene un estado compartido.

Elemento	Definición
Usuarios principales	Estudiantes y docentes; una cuenta distinta por participante en red.
Valor esperado	Interacción en tiempo real y evidencia práctica de POO, capas, concurrencia y persistencia.
Modalidad principal del proyecto	Batalla Azul contra Rojo con seis clientes y selección de avión Ligero o Pesado.
Funciones complementarias	Modo Solo, duelo local, registro de resultados y ranking de partidas locales.
Criterio de aceptación propuesto	Seis usuarios completan una batalla y el resultado queda guardado con sus seis detalles.

Modelo de sistema y variables

Frontera del sistema

El sistema comprende el cliente Windows, el servidor de cuentas y salas, el motor de combate y MySQL. Los jugadores generan las entradas; la red y el equipo de ejecución condicionan los tiempos de comunicación. El docente participa en la revisión y aceptación académica.



Modelo conceptual de niveles y flujos; no es una medición del sistema.

Figura 1. Modelo conceptual de niveles y flujos.

Variable	Tipo / unidad	Interpretación
Q_A, Q_R	Nivel; jugadores	Espera de Azul y Rojo por cola. Se distinguen las colas local y red.
S	Nivel; salas	Salas que ya completaron seis participantes y ejecutan el combate.
H_i	Estado; puntos de vida	Vida del piloto i: inicial 20 en Ligero o 30 en Pesado.
X_i, Y_i	Estado; unidades del mapa	Posición validada por el servidor en el área de 560 × 640.
B	Nivel; proyectiles	Cantidad de balas activas; máximo global de 24 por batalla.
D_i, A_i, K_i	Acumulados; conteos	Disparos, aciertos y derribos de cada piloto.
R	Acumulado; partidas	Resultados 3v3 persistidos junto con seis detalles.
λ, μ, δ	Flujos; eventos/s	Llegadas, formación de salas y salidas de la espera.
T_tick, bytes_f	Indicadores; ms / bytes	Duración observada de ciclo y tamaño del cuadro JSON.

Evidencia: TeamBattle.cs; SalasEnRed.cs; PartidaEquipo.cs. λ, μ y δ son variables de análisis propuestas, no contadores ya instrumentados.

Relaciones causales y retroalimentación

Relaciones causales

Los signos expresan el efecto esperado de aumentar una variable, manteniendo las demás condiciones constantes. Son hipótesis de análisis basadas en la lógica del sistema; no constituyen resultados de una simulación calibrada.

Origen → efecto	Signo	Fundamento y condición
Llegadas → jugadores en espera	+	Los usuarios autenticados que buscan partida ingresan a la cola.
Formación de salas → espera por equipo	−	Cada sala consume tres jugadores azules y tres rojos.
Esperas de ambos equipos → salas posibles	+	La cantidad posible está limitada por el equipo menos numeroso.
Disparos válidos → balas activas	+	Se agregan balas si terminó el enfriamiento y B < 24.
Impactos enemigos → vida	−	Cada impacto confirmado resta diez puntos de vida.
Vida agotada → aviones vivos	−	Un piloto con vida cero deja de actuar y de dibujarse.
Eliminación del equipo → partidas terminadas	+	El combate finaliza cuando un equipo ya no tiene pilotos vivos.
Finalizaciones guardadas → historial	+	Se almacena una cabecera y el detalle de los seis pilotos.
Salas simultáneas → trabajo del servidor	+	Hipótesis a medir: crecen actualizaciones, envíos y tareas concurrentes.

Retroalimentación de balance

B1 — Capacidad de proyectiles: más balas activas reducen el margen disponible hasta el máximo de 24; al alcanzar el límite se bloquean nuevas balas, y su eliminación por impacto o salida del mapa vuelve a liberar capacidad. Es una regulación explícita de la simulación.

B2 — Vida y capacidad ofensiva: más impactos reducen la vida; al llegar a cero, el piloto deja de moverse y disparar. Disminuye así la generación futura de acciones por parte de ese avión. La ventaja entre equipos puede amplificarse a medida que uno pierde participantes; esta interpretación debe contrastarse con partidas registradas.

Retardos relevantes

El enfriamiento entre disparos, la espera hasta completar 3 + 3, la transmisión de cuadros y la escritura final en MySQL introducen retardos distintos. El intervalo programado de un tick no equivale al tiempo de respuesta que percibe el jugador.

Modelo dinámico e indicadores

Modelo dinámico simplificado

Para un intervalo Δt, una cola y un equipo e: $Q_e(t + \Delta t) = Q_e(t) + llegadas_e - 3 \cdot salas_formadas - salidas_e$. Las cantidades de llegadas y salidas corresponden al intervalo. El número de nuevas salas no puede superar $\min(\lfloor Q_A/3 \rfloor, \lfloor Q_R/3 \rfloor)$.

En el combate: $X_i(t + \Delta t) = limitar(X_i + v_i \cdot u_x \cdot \Delta t)$ y $Y_i(t + \Delta t) = limitar(Y_i + v_i \cdot u_y \cdot \Delta t)$, solo si la posición candidata no se cruza con obstáculos ni otros pilotos vivos. La dirección diagonal se normaliza. El paso utilizado en red es $\Delta t = 1/30$ s.

$H_i(t + \Delta t) = \max(0, H_i(t) - 10 \cdot impactos_i)$. Una desconexión fija la vida en cero. $B(t + \Delta t) = B(t) + disparos_aceptados - balas_eliminadas$, con $0 \le B \le 24$. El combate termina cuando los tres pilotos de algún equipo tienen vida cero.

Parámetro del modo red	Valor en código
Mapa; obstáculos	560 × 640; dos rectángulos de 40 × 40.
Avión Ligero	Vida 20; velocidad 255 unidades/s; enfriamiento 0,25 s.
Avión Pesado	Vida 30; velocidad 190 unidades/s; enfriamiento 0,38 s.
Bala	Velocidad vertical 410 unidades/s; daño 10; sin fuego amigo.

Indicadores de desempeño propuestos

Indicador	Cálculo / objetivo	Situación
Ticks efectivos	Ticks ejecutados / tiempo real; objetivo ≥ 30/s.	Timer configurado; medición pendiente.
Cuadro periódico	Bytes UTF-8 del JSON f; máximo 1024.	Control explícito en servidor; falta ensayo sostenido.
Espera de emparejamiento	Inicio de partida − entrada en cola, p50/p95.	Falta instrumentación.
Persistencia completa	Partidas con seis detalles / partidas finalizadas.	Existe guardado; falta integración.
Resistencia a desconexión	Sala continúa tras perder un cliente.	Prueba de Core localizada; ensayo de red pendiente.
Tiempo local de actualización	Promedio de 60 muestras de game.Update.	Instrumentado en FlightCanvas; sin valor medido aquí.

El límite de 1024 bytes se comprueba sobre el contenido JSON del mensaje f; no incluye cabeceras WebSocket/TCP/IP ni demuestra latencia. Los mensajes start, lobby y end no tienen esa comprobación.

Planificación — Sprint 0

Roles del equipo Scrum

Asignación propuesta para organizar el trabajo; los nombres y la aceptación de los roles deben registrarse con el equipo. La plantilla identifica al docente como Product Owner. “Sprint” se interpreta como el período de trabajo y no como un cargo Scrum.

Rol / responsabilidad	Trabajo y entregable
Product Owner: docente	Priorizar, aclarar reglas y validar alcance y aceptación.
Facilitación Scrum: por designar	Coordinar reuniones, impedimentos y revisión del incremento.
Análisis de requisitos	Mantener SRS, backlog, trazabilidad y decisiones pendientes.
Frontend	Acceso, selección, lobby, controles y renderizado Windows Forms.
Backend	Sesiones, WebSocket, salas, coordinación y persistencia final.
Arquitectura y base de datos	Revisar capas, clases, esquema MySQL e inicialización.
Calidad y pruebas	Validar reglas, red, fallos, rendimiento y evidencias de aceptación.

Backlog inicial actualizado

ID / prioridad	Historia o actividad	Estado documental
HU01 / Alta	Como jugador, crear cuenta e ingresar.	Código disponible; falta prueba integrada.
HU02 / Alta	Como jugador, elegir equipo y avión y esperar 3v3.	Código disponible; grupos preformados pendientes.
HU03 / Alta	Como jugador, moverme y disparar con estado compartido.	Código disponible; falta prueba con seis clientes.
HU04 / Alta	Como jugador, ver el ganador y conservar estadísticas.	Código disponible; falta verificar MySQL al finalizar.
HU05 / Alta	Como equipo, validar desconexiones y continuidad.	Prueba de dominio localizada; falta ensayo de red.
HU06 / Media	Medir 30 ticks/s, bytes y varias salas.	Instrumentación y medición pendientes.
HU07 / Media	Completar capturas, miembros y revisión académica.	Pendiente.
HU08 / Media	Resolver diferencias entre SRS e implementación.	Transporte, grupos, orden de selección y cifrado.

Criterio propuesto de terminado: código revisado, caso de aceptación ejecutado, evidencia registrada y documentación actualizada. La inspección estática por sí sola no cierra una historia.

Requisitos funcionales — acceso y sala

Se conservan los identificadores de la SRS proporcionada. Los estados indican evidencia de implementación, no aprobación de pruebas de aceptación.

RF01 — Gestión de cuentas

Crear usuario e iniciar sesión contra MySQL; sesión Bearer para acceder al juego.

Estado: Implementado en código.

Evidencia: UsuarioService, ClaveHasher, UsuariosEndpoints, SesionService.

RF02 — Conexión de clientes

Aceptar clientes concurrentes y mantener comunicación de juego. Se utiliza WebSocket sobre TCP, no un listener TCP/UDP propio.

Estado: Implementado con adaptación.

Evidencia: Program: /ws/juego; SalasEnRed.AtenderAsync.

RF03 — Cola de espera

Mantener espera y publicar ocupación de cada equipo antes de iniciar.

Estado: Implementado con adaptación.

Evidencia: SalasEnRed.espera, PublicarEsperaAsync; colas local y red separadas.

RF04 — Emparejamiento

Iniciar únicamente con tres usuarios distintos por bando. La SRS también exige preservar grupos preformados de dos o tres.

Estado: Parcial.

Evidencia: FormarSalas toma tres por equipo elegido; no existe identidad ni gestión de grupos.

RF05 — Selección de avión

Elegir Ligero o Pesado. En el código la selección ocurre antes de buscar partida; la SRS la ubica después de completar equipos.

Estado: Implementado con diferencia de flujo.

Evidencia: NetworkForm: ComboBox de avión y parámetro avion en /ws/juego.

Requisitos funcionales — combate y resultados

Se conservan los identificadores de la SRS proporcionada. Los estados indican evidencia de implementación, no aprobación de pruebas de aceptación.

RF06 — Movimiento

Capturar flechas o WASD, enviar dirección y validar posición en el servidor.

Estado: Implementado en código.

Evidencia: NetworkForm.EnviarControlesAsync; TeamBattle.RecibirEntrada y Actualizar.

RF07 — Disparos

Enviar intención al pulsar espacio; generar balas en servidor con enfriamiento y trayectoria.

Estado: Implementado en código.

Evidencia: TeamBattle.Actualizar; BalaBatalla.

RF08 — Colisiones

Comprobar límites, obstáculos y otros pilotos. Las balas dañan únicamente al equipo rival.

Estado: Implementado en código.

Evidencia: TeamBattle: RectangleF.IntersectsWith, Math.Clamp y filtros de equipo.

RF09 — Sincronización

Enviar estado de los seis pilotos y balas a todos los clientes de la sala.

Estado: Implementado en código.

Evidencia: SalaRed.EjecutarAsync: mensaje f; NetworkForm: recepción y dibujo.

RF10 — Victoria

Finalizar cuando un equipo ya no tenga pilotos con vida. Una desconexión elimina su piloto.

Estado: Implementado en código.

Evidencia: TeamBattle.RevisarGanador y Desconectar.

RF11 — Resultados

Guardar equipo ganador, duración y estadísticas de los seis participantes en MySQL.

Estado: Implementado en código.

Evidencia: PartidaEquipo.DesdeBatalla; ResultadoEquipoRepository.GuardarAsync.

Requisitos no funcionales

La implementación aporta mecanismos de calidad, pero su cumplimiento debe demostrarse mediante ejecución y mediciones. Esta revisión no certifica conformidad con una norma externa.

RNF01 — Rendimiento

Procesar y publicar al menos 30 ticks/s. PeriodicTimer a 1/30 s. El envío se espera en cada vuelta; un cliente lento puede afectar el ritmo. Medición bajo carga pendiente.

RNF02 — Tamaño del estado

Cuadro periódico de hasta 1024 bytes. JSON compacto, coordenadas enteras, máximo 24 balas y comprobación de bytes UTF-8. Validar el límite en todos los escenarios; precisar si el requisito incluye cabeceras.

RNF03 — Autoridad

Servidor decide vida, daño y colisiones en red. TeamBattle corre en SalaRed; cliente solo envía dirección y disparo. Implementado para 3v3; los modos locales simulan en cliente.

RNF04 — Protección de contraseñas

Almacenar hash resistente de contraseña. PBKDF2-SHA256; 150 000 iteraciones, sal de 16 bytes, salida de 32 bytes. Implementado. Hash y cifrado son conceptos distintos; el transporte HTTP no cifra credenciales.

RNF05 — Mantenibilidad

Separar responsabilidades y proyectos. Cliente, Core, Contracts, Data, Servidor y Tests separados. Implementado; revisar acoplamiento y documentación en cada cambio.

RNF06 — Fiabilidad

La caída de un cliente no debe terminar el juego para los demás. Se capturan errores de socket y se llama a Desconectar; vida queda en cero. Prueba de dominio presente; faltan cortes reales y casos de cliente bloqueado.

RNF07 — Compatibilidad

Ejecutar cliente en Windows. Cliente net10.0-windows con UseWindowsForms. Requiere verificación de ejecución y publicación en Windows.

Arquitectura y casos de uso

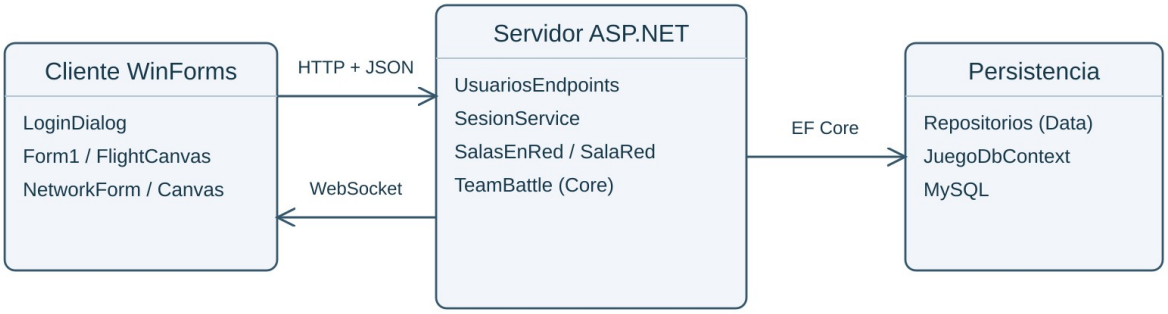


Figura 2. Arquitectura del modo en red. Los modos locales ejecutan FlightGame en el cliente.

Diagrama de casos de uso



Figura 3. Casos principales del jugador en una batalla 3v3.

Actor principal: jugador. El servidor y MySQL son componentes internos, no usuarios humanos. El flujo exige sesión válida y cuenta libre; no inicia el combate hasta completar ambos equipos. El registro del resultado se ejecuta como parte de la finalización.

Evidencia: Program.cs; UsuariosEndpoints.cs; NetworkForm.cs; SalasEnRed.cs.

Diagramas de clases

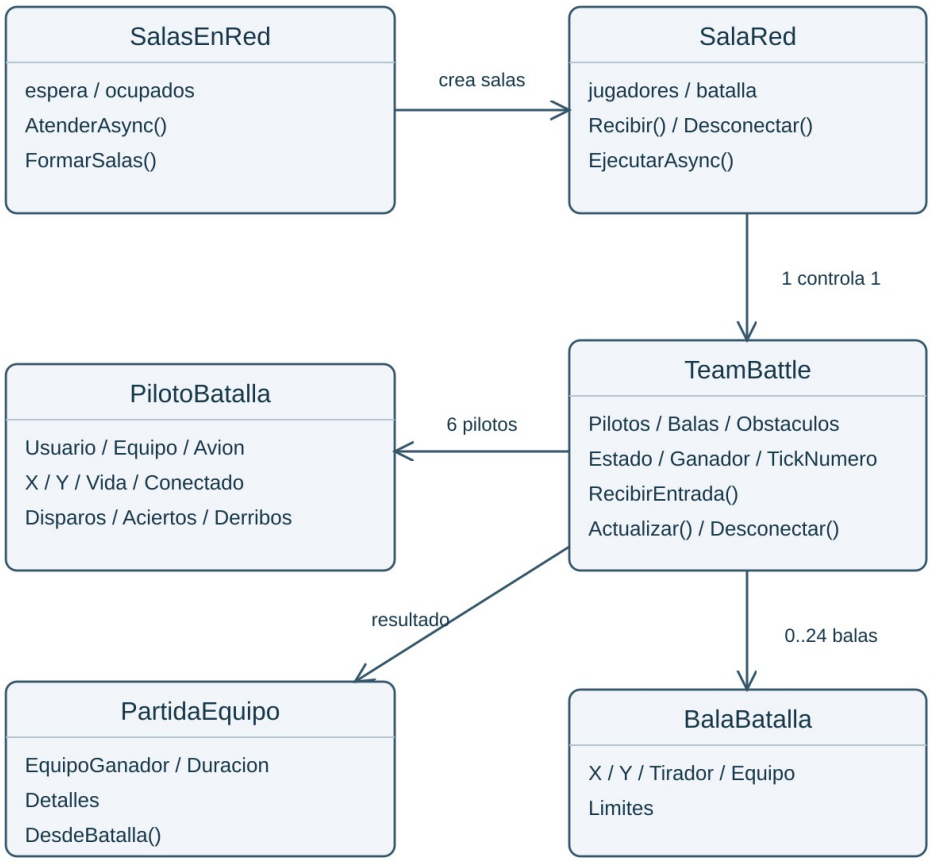


Figura 4. Vista de clases de red. Las flechas rotuladas indican uso o composición conceptual.

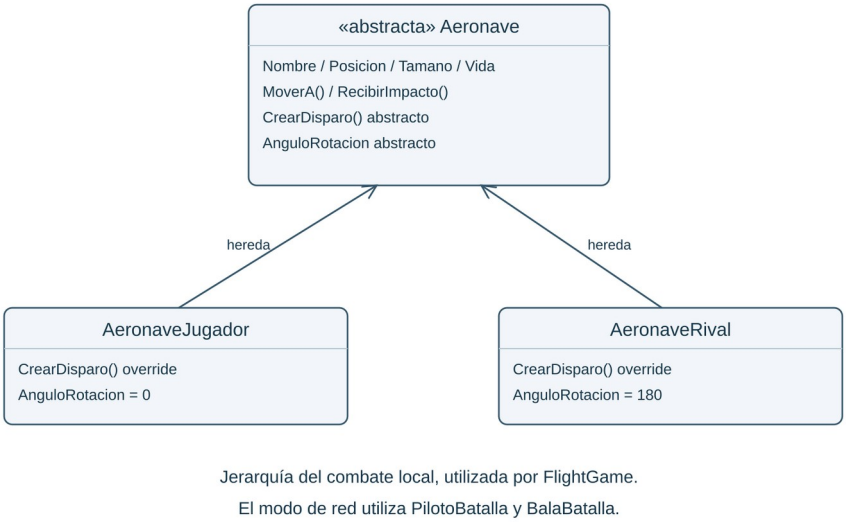


Figura 5. Herencia y polimorfismo de las aeronaves del modo local.

La red usa PilotoBatalla y BalaBatalla como representación de sus entidades. El diagrama evita atribuir a estas clases una herencia de Aeronave que no aparece en el código.

Diagrama de secuencia

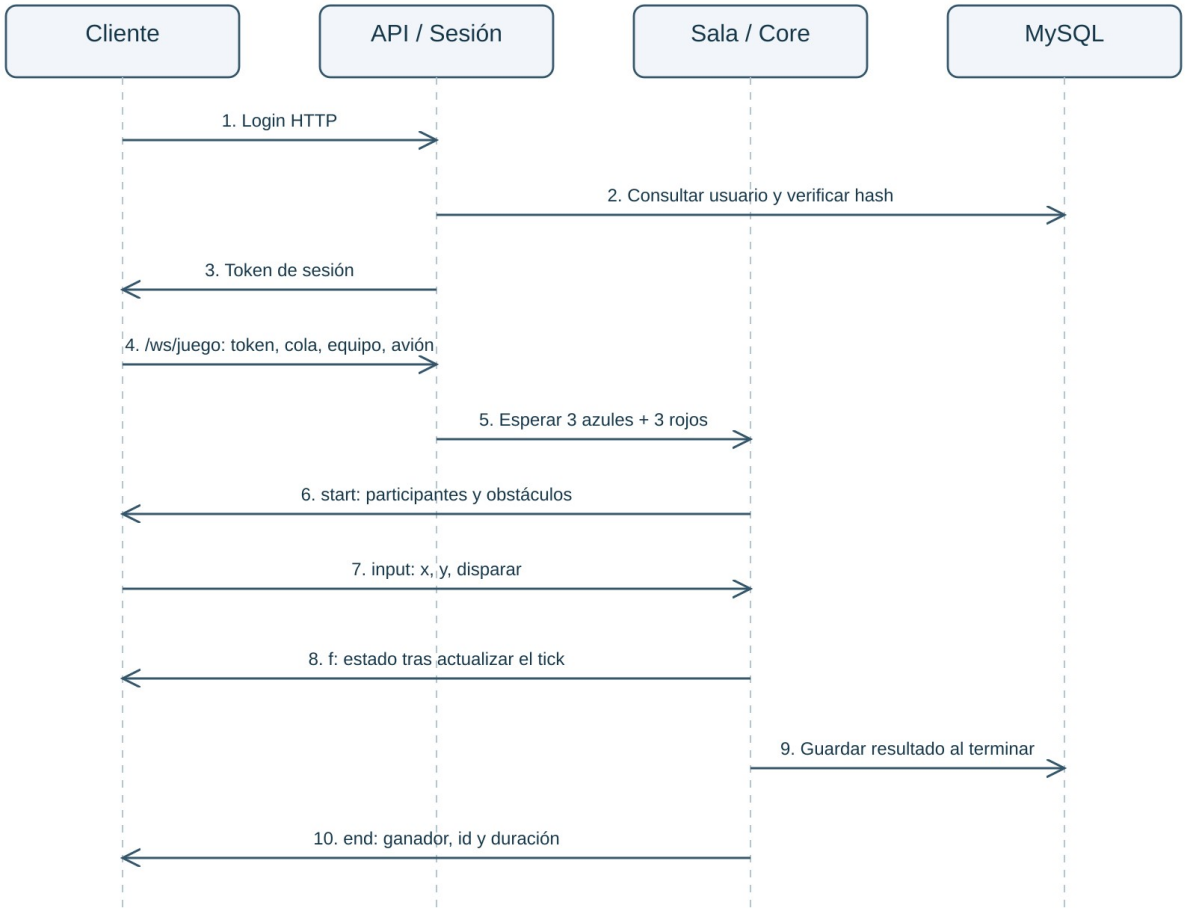


Figura 6. Secuencia simplificada del acceso, combate y persistencia.

Excepciones y variantes

Una sesión inválida produce HTTP 401 antes de aceptar el WebSocket. Una cuenta ya ocupada recibe un mensaje error. Si se desconecta un participante, el servidor retira su capacidad de combate. Si el guardado final falla, la excepción queda en el log; no se observa una estrategia de reintento o una cola durable de resultados.

En el paso 2 se recupera el usuario de MySQL y el servidor verifica el hash; la verificación criptográfica no la realiza la base de datos. El diagrama agrupa estas operaciones para facilitar la lectura.

Evidencia: UsuariosEndpoints.cs; UsuarioService.cs; SalasEnRed.cs; ResultadoEquipoRepository.cs.

Base de datos y entidad-relación

Clase de acceso y creación del esquema

JuegoDbContext deriva de DbContext y configura cuatro conjuntos: Partidas, Usuarios, PartidasEquipo y DetallesJugadorPartida. InicializadorMySql ejecuta EnsureCreated y sentencias CREATE TABLE IF NOT EXISTS; también incorpora Modo y Oponente en tablas de partidas anteriores si esas columnas faltan.

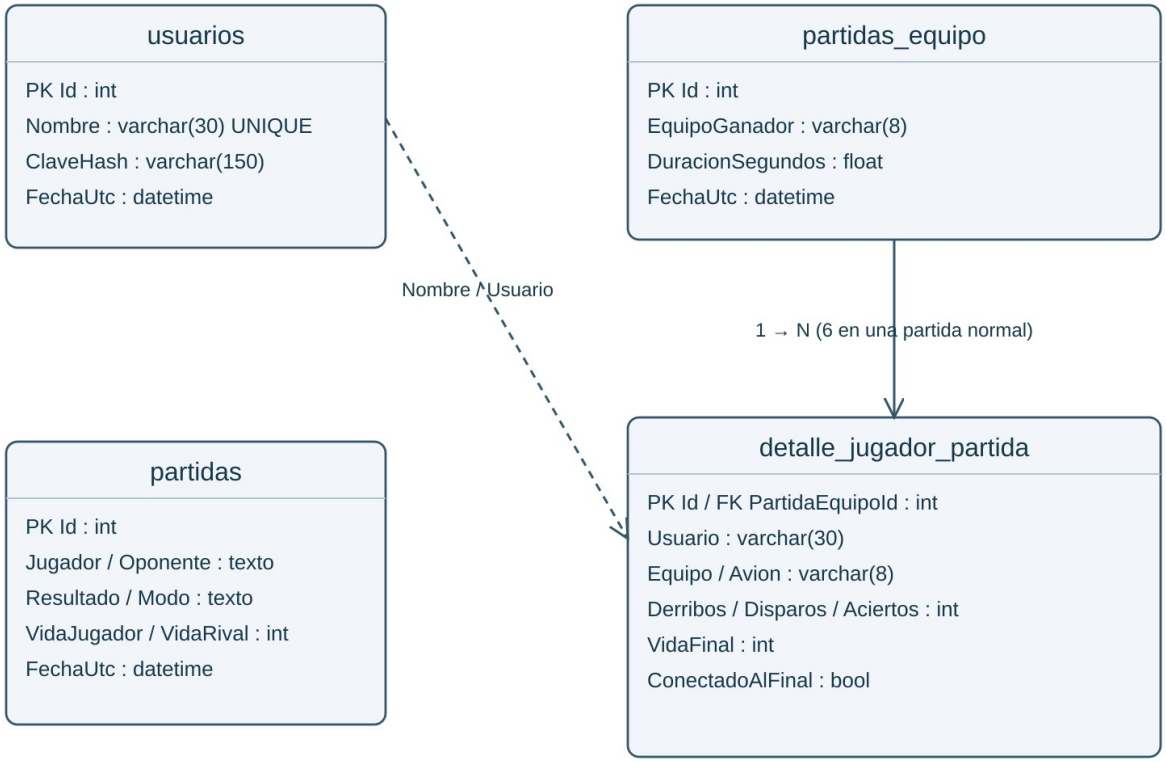


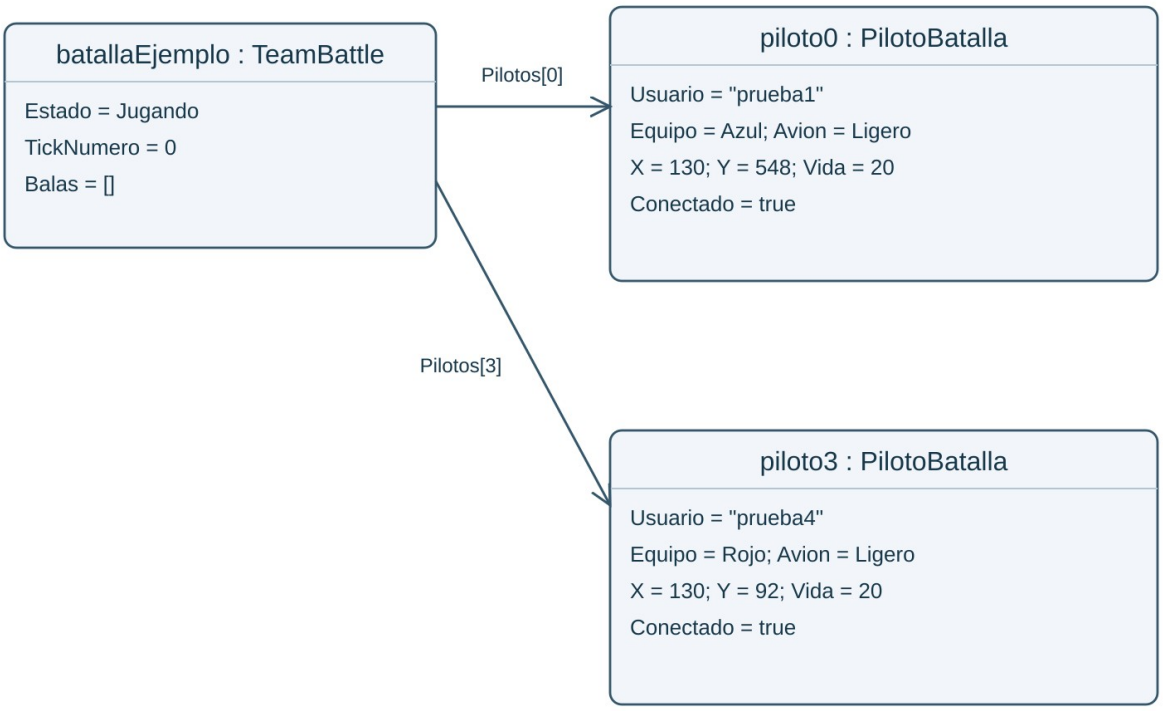
Figura 7. Esquema físico simplificado y vínculo lógico de usuarios.

Existe una clave foránea desde detalle_jugador_partida.PartidaEquipoId hacia partidas_equipo.Id. Usuario se guarda como nombre textual; no hay una clave foránea al identificador de usuarios. La relación entre usuario y resultados es lógica y debe documentarse así.

ResultadoEquipoRepository agrega PartidaEquipo con sus detalles y ejecuta SaveChangesAsync. Los resultados locales se almacenan en partidas mediante PartidaRepository. El endpoint de ranking consulta esas partidas; no se debe afirmar que ya incluya las batallas 3v3.

Evidencia: JuegoDbContext.cs; InicializadorMySql.cs; ResultadoEquipoRepository.cs; PartidaRepository.cs.

Diagrama de objetos



Ejemplo derivado del constructor: existen otros cuatro pilotos en la misma batalla.

Figura 8. Instancia ilustrativa del estado inicial de una batalla.

Interpretación del ejemplo

El constructor asigna tres pilotos a cada bando. Las posiciones X son 130, 280 y 430; la fila Azul inicia en Y = 548 y la Roja en Y = 92. Un Ligero comienza con 20 puntos de vida y un Pesado con 30. El ejemplo representa el estado que deriva del código; no es una captura de una ejecución real.

Invariantes del sistema

- Exactamente seis participantes y tres por equipo al construir TeamBattle.
- Los seis nombres de usuario deben ser distintos, sin distinguir mayúsculas y minúsculas.
- La entrada horizontal y vertical se limita al rango −1 a 1.
- La vida no puede descender por debajo de cero al recibir un impacto.
- El máximo de balas activas se aplica a toda la batalla.
- Tras terminar, Actualizar deja de avanzar la simulación.

Evidencia: TeamBattle.cs: constructor, RecibirEntrada, Actualizar y RevisarGanador.

Referencias y entregables

Referencias documentales

1. Plantilla académica: “Documentación Proyecto.docx.pdf”, proporcionada en index_qualitysoft. Fuente de la estructura de este documento.

2. Especificación: “Documentación JUEGO DE AERONAVES.docx.pdf”, proporcionada en index_qualitysoft. Fuente de RF01–RF11 y RNF01–RNF07.

3. Código local: /home/joel/Work/juego-de-aviones, revisado el 26/10/2026. Repositorio remoto: https://github.com/slim-carrasco/juego-de-aviones. Base Git 074a2ccd393f2a0aa2ab7339544d0f1fd24cec2c más cambios locales.

4. README.md del repositorio: ejecución, distribución, cuentas de demostración y arquitectura.

5. docs/EXPOSICION-5-PERSONAS.md y docs/DISENO-AVION.md: guía de exposición y recurso gráfico. La guía contiene referencias históricas a pruebas y modos locales; no se usa como evidencia de la validación actual.

6. Archivos fuente y pruebas citados en cada sección. Las rutas abreviadas se resuelven mediante el inventario al final del documento.

Entregables del proyecto

Entregable de la plantilla	Avance incluido	Pendiente
1. Hoja de identificación	Anexo 1: propósito, alcance, tecnologías y actores.	Integrantes y aprobación.
2. Hoja de diseño técnico	Anexo 2: componentes, contratos y despliegue.	Pruebas integradas del entorno.
3. Requisitos funcionales	RF01–RF11 y matriz en Anexo 3.	Aceptar diferencias y ejecutar casos.
4. Requisitos no funcionales	RNF01–RNF07 y matriz en Anexo 4.	Mediciones y evidencias.
5. Visión y funcionamiento	Anexo 5: recorrido de usuario y plan de capturas.	Capturas reales desde Windows.

El PDF original se conserva sin cambios. El avance se entrega en Word editable y PDF, con fuentes reproducibles y diagramas SVG/PNG dentro de docs/documentacion-proyecto/.

Anexo 1. Identificación del proyecto

Campo	Contenido
Nombre	Juego de aeronaves
Problema que aborda	Coordinar interacción simultánea y un estado consistente de combate en una aplicación de aprendizaje.
Propósito	Desarrollar y documentar una simulación de combate aéreo con clientes Windows y autoridad central en el modo en red.
Beneficiarios	Estudiantes y docentes que utilizan el juego o analizan su construcción.
Alcance funcional	Cuentas; Solo; Duelo local; combate 3v3; selección de avión y equipo; lobby; colisiones; victoria; resultados.
Tecnologías	C#; .NET 10; Windows Forms; ASP.NET Core; HTTP/JSON; WebSocket; Entity Framework Core; MySQL; xUnit.
Repositorio	https://github.com/slim-carrasco/juego-de-aviones
Ruta de desarrollo	/home/joel/Work/juego-de-aviones, enlace a /home/joel/Windows/juego-de-aviones.
Acceso desde Windows	Z:\juego-de-aviones o ruta compartida descrita en README.
Responsable académico	Docente del curso, según identificación de la plantilla.
Integrantes	Por completar con nombres y responsabilidades reales.
Estado	Avance implementado en copia local; aceptación y mediciones pendientes.
Exclusiones del avance	No se aportan evidencia de producción, capturas reales ni cifras de rendimiento medidas.
Restricciones	Cliente Windows; MySQL disponible al arrancar servidor; seis cuentas diferentes para 3v3.

Decisiones por confirmar con el equipo

Aceptar WebSocket como transporte del juego; decidir si se conserva la exigencia de grupos preformados de la SRS; confirmar selección de avión antes del lobby; definir el alcance exacto del límite de bytes y la evaluación de rendimiento.

Anexo 2. Hoja de diseño técnico

Proyecto / componente	Responsabilidad
JuegoAviones (raíz)	WinForms: acceso, menú, FlightCanvas local y NetworkForm en red.
JuegoAviones.Core	Entidades, validaciones, FlightGame, TeamBattle e interfaces de repositorios.
JuegoAviones.Contracts	DTO de credenciales, sesiones, partidas y ranking HTTP.
JuegoAviones.Data	EF Core, inicialización MySQL y repositorios.
JuegoAviones.Servidor	API, sesiones en memoria, salas y bucle autoritativo.
JuegoAviones.Tests	Pruebas de dominio de partidas, juego local y batalla por equipos.

Interfaces principales

Interfaz	Propósito / contrato
POST /api/usuarios/registrar; /login	Credenciales → nombre, token y vencimiento.
GET /api/usuarios/yo	Verificación de la sesión Bearer.
WebSocket /ws/juego	Parámetros cola, equipo, avion; cabecera Authorization Bearer.
Cliente → servidor	JSON input: {t: "input", x: −1..1, y: −1..1, f: booleano}.
Servidor → cliente	lobby: espera; start: configuración; f: posiciones y balas; end: resultado; error: rechazo.
POST /api/partidas; /duelo	Registro de resultado de las modalidades locales con sesión.
GET /api/partidas/ranking; /{id}	Ranking y consulta de los registros locales.
GET /health	Estado HTTP del proceso; no sustituye una comprobación de escritura en MySQL.

Despliegue de desarrollo

infra/docker-compose.yml prepara MySQL. iniciar-servidor.bat inicia la API en la VM y ejecutar.bat abre el cliente. publicar-cliente.bat prepara la distribución Windows. El cliente requiere la dirección de un servidor compartido para jugar desde otros equipos.

La configuración de conexión se obtiene de ConnectionStrings:Juego; el proveedor estándar permite sobrescribirla con ConnectionStrings__Juego. No se reproducen credenciales de configuración en este documento. Las sesiones viven en memoria y vencen a las doce horas; un reinicio del servidor las pierde.

Anexo 3. Trazabilidad funcional

Casos de aceptación propuestos. “Por ejecutar” significa que en esta revisión no se realizó el ensayo integrado. La existencia de una prueba en el repositorio no equivale a una ejecución aprobada.

RF	Caso y resultado esperado	Evidencia / estado
RF01	Registrar nombre nuevo; ingresar; rechazar contraseña incorrecta y duplicado.	UsuarioService / UsuariosEndpoints. Por ejecutar con MySQL.
RF02	Abrir seis clientes distintos con sesión válida; todos reciben mensajes.	Program / SalasEnRed. Por ejecutar.
RF03	Con cinco usuarios el lobby informa espera y no inicia; el sexto completa la sala.	PublicarEsperaAsync / FormarSalas. Por ejecutar.
RF04	3 azules + 3 rojos inician; un séptimo espera otra sala; grupos 2+1 y 3 se conservan.	FormarSalas y constructor TeamBattle. Grupos no implementados; resto por ejecutar.
RF05	Ligero y Pesado elegidos se reflejan en vida y parámetros iniciales.	NetworkForm / TeamBattle. Por ejecutar; aceptar diferencia de orden.
RF06	Mantener dirección y diagonal; no salir del mapa ni atravesar obstáculos.	Test ElServidorMueveYLimitaLosAviones localizado. Ejecución pendiente.
RF07	Mantener espacio; respetar enfriamiento y máximo de balas.	TeamBattle.Actualizar. Falta caso específico de red.
RF08	Disparar contra enemigo, aliado y obstáculo; solo daño enemigo.	TeamBattle.Actualizar. Falta prueba completa de colisiones.
RF09	Comparar estado de la misma sala en seis clientes y aislamiento entre salas.	SalaRed / NetworkForm. Por ejecutar.
RF10	Eliminar tres rivales; declarar ganador una vez y detener simulación.	Test TresDerribosTerminanElCombateSinInputsDelRival localizado. Ejecución pendiente.
RF11	Finalizar y consultar una cabecera con seis detalles correctos en MySQL.	PartidaEquipo / ResultadoEquipoRepository. Por ejecutar.

Cobertura funcional complementaria

FlightGameTests contiene casos de pausa, movimiento, enfriamiento, victoria, derrota, tiempos inválidos y duelo local. PartidaTests comprueba validaciones de los registros. No se localizó una suite de integración para endpoints, WebSocket y MySQL.

Anexo 4. Validación no funcional

RNF	Procedimiento propuesto	Evidencia que debe registrarse
RNF01	Ejecutar partidas y varias salas; medir intervalos reales y tiempo de actualización y envío.	Ticks/s por sala; percentiles de ciclo; carga y características del equipo. Pendiente.
RNF02	Registrar bytes UTF-8 del mensaje f con 6 pilotos y hasta 24 balas.	Máximo observado y número de muestras; distinguir payload y tráfico total. Pendiente.
RNF03	Enviar direcciones fuera de rango y comprobar límites; intentar alterar vida desde cliente.	Entradas rechazadas o normalizadas y estado del servidor. Pendiente.
RNF04	Crear cuenta, inspeccionar formato de hash y verificar acceso correcto e incorrecto.	PBKDF2 con sal; ninguna contraseña en texto plano en la tabla. Pendiente de integración.
RNF05	Compilar solución e inspeccionar dependencias entre proyectos.	Salida de compilación y dependencias. Pendiente de SDK .NET 10 en este entorno.
RNF06	Cerrar un cliente; cortar conexión; mantener cliente sin consumir datos.	Resto de sala continúa; resultado coherente; límites temporales de recuperación. Pendiente.
RNF07	Publicar cliente y abrirlo en Windows; comprobar Solo, Duelo local y 3v3.	Versión de Windows, artefacto usado y capturas reales. Pendiente.

Observaciones técnicas para la evaluación

El bucle de sala espera la finalización de los envíos a todos los jugadores. Sin una medición y una política de tiempo máximo por envío, no puede garantizarse la tasa efectiva ante un receptor lento. El tamaño de un mensaje tampoco garantiza baja latencia.

El formato de contraseñas usa PBKDF2-SHA256 con sal aleatoria y comparación de tiempo constante. El acceso HTTP/WS del entorno de desarrollo no aporta cifrado de transporte. Si el alcance exige comunicación cifrada, debe configurarse y validarse HTTPS/WSS.

El caso de desconexión de TeamBattle verifica la regla de dominio; no demuestra por sí solo tolerancia a errores del socket, timeout ni estabilidad del proceso servidor.

Anexo 5. Visión y funcionamiento

Recorrido del usuario

1. Abrir JuegoAviones e indicar el servidor disponible. Iniciar sesión o crear una cuenta.

2. Elegir Solo, Duelo local o 3 vs 3. Para una batalla entre equipos, cada usuario usa su propia cuenta.

3. En 3 vs 3 entre laptops, seleccionar Ligero o Pesado y Equipo 1 (Azul) o Equipo 2 (Rojo); pulsar Buscar partida.

4. Esperar mientras el lobby muestra la ocupación. El servidor inicia cuando dispone de tres jugadores por bando dentro de la misma cola.

5. Moverse con flechas o WASD y disparar con espacio. El servidor calcula las posiciones y el cliente dibuja el cuadro recibido.

6. Cuando un bando pierde sus tres aviones, ver el ganador. El servidor guarda los resultados y envía el identificador de partida.

7. Para demostrar seis jugadores en una sola PC, usar 3 vs 3 · esta PC. Esa opción abre seis ventanas con cuentas de prueba y utiliza una cola separada.

Capturas requeridas para completar este anexo

Código	Pantalla real que debe adjuntarse	Qué documenta
CAP01	Acceso y registro, ocultando contraseñas y tokens.	Inicio de sesión y mensajes de validación.
CAP02	Menú con las tres modalidades.	Alcance disponible al usuario.
CAP03	Lobby antes de completar 3 + 3.	Selección de avión, equipo y espera.
CAP04	Batalla con seis clientes activos.	Posiciones, vidas, obstáculos y proyectiles.
CAP05	Fin de partida con ganador e identificador.	Finalización comunicada al jugador.
CAP06	Consulta MySQL de cabecera y seis detalles.	Persistencia del resultado 3v3.

Las capturas permanecen pendientes: no se ejecutó WinForms en esta revisión desde Linux. Los diagramas del documento son modelos del código y no sustituyen fotografías o capturas de ejecución.

Estado de revisión y próximos pasos

Validación realizada

Se inspeccionaron la estructura de proyectos, los endpoints HTTP y WebSocket, la formación de salas, el motor TeamBattle, las sesiones, el hash de contraseñas, la inicialización MySQL, los repositorios y los archivos de pruebas. No se modificó la lógica del juego para elaborar este avance.

Se intentó ejecutar dotnet test con artefactos en una carpeta temporal. El entorno informó “No .NET SDKs were found”. Por ello no se reporta ninguna prueba como aprobada ni una compilación verificada en esta revisión. Tampoco se ejecutaron seis clientes ni se consultó una base de datos en funcionamiento.

Diferencias y pendientes prioritarios

Prioridad	Acción
Alta	Revisar RF04: no hay gestión explícita de grupos preformados de dos o tres jugadores.
Alta	Ejecutar en un entorno con SDK .NET 10 las pruebas existentes y los casos de aceptación con seis clientes.
Alta	Verificar ganador, duración y seis detalles en MySQL tras una batalla completa.
Media	Medir ticks/s reales, tamaño de los cuadros, estabilidad y comportamiento ante clientes lentos.
Media	Confirmar con el docente transporte WebSocket y selección de avión anterior al emparejamiento.
Media	Actualizar la guía de exposición: contiene descripciones del combate local y cifras históricas de pruebas inconsistentes.
Media	Completar integrantes, responsables, capturas y acta de aceptación.

Inventario de evidencia principal

Raíz: README.md, JuegoAviones.csproj, NetworkForm.cs, LoginDialog.cs y Form1.cs. Core: Gameplay/TeamBattle.cs, Gameplay/FlightGame.cs, CasosDeUso/UsuarioService.cs, CasosDeUso/ClaveHasher.cs y Entidades/PartidaEquipo.cs. Servidor: Program.cs, Red/SalasEnRed.cs, Red/UsuariosEndpoints.cs, Red/PartidasEndpoints.cs y Sesiones/SesionService.cs. Data: Conexion/JuegoDbContext.cs, Conexion/InicializadorMySql.cs y Repositorios/ResultadoEquipoRepository.cs. Tests: TeamBattleTests.cs, FlightGameTests.cs y PartidaTests.cs. Cada ruta de Core, Servidor, Data o Tests se encuentra bajo JuegoAviones.<nombre>/.